

ASSESSMENT TOOL – 1

Course Code	CSA0702	Course Title	Computer Networks
CO Assessed	CO4	Assessment	Coding Challenge
Weightage		Total Marks	

Student Name	Cindy R	Reg. No.	192412571
Section / Batch	ECE-2024	Date of Submission	14/08/2026
Faculty Incharge	Dr R S Selvan	Assessment	Individual Submission

Total Marks		Obtained Marks	
-------------	--	----------------	--

Code of Conduct:

I Cindy R/192412571 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. Star Topology Diagram Generator

Generates an ASCII star topology with a central switch, host labels, and Cat6 UTP cable lengths.

1_star_topology.py

```
def create_star_topology(n):
```

```
    """Generate an ASCII star topology for n hosts."""
```

```
    if n <= 0:
```

```
        print("Error: Number of hosts must be greater than 0.")
```

```
        return
```

```
    switch_name = "SWITCH"
```

```
    print("\n----- STAR TOPOLOGY ----- \n")
```

```
    print("        [", switch_name, "]")
```

```
    print("        |")
```

```
    for i in range(1, n + 1):
```

```
        length = 10 + (i * 5) # Example cable length
```

```
        print(f"        |---- Cat6 UTP: {length} m ---- HOST{i} ")
```

```
    print("\nCentral Switch :", switch_name)
```

```
    print("Number of Hosts:", n)
```

```
    print("----- \n")
```

```
# Test
```

```
try:
```

```
    n = int(input("Enter number of hosts: "))
```

```
    create_star_topology(n)
```

```
except ValueError:
```

```
    print("Error: Please enter a valid integer.")
```

Example Output

===== STAR TOPOLOGY =====

[SWITCH]

|
|---- Cat6 UTP: 15 m ---- HOST1

|---- Cat6 UTP: 20 m ---- HOST2

|---- Cat6 UTP: 25 m ---- HOST3

|---- Cat6 UTP: 30 m ---- HOST4

Central Switch : SWITCH

Number of Hosts: 4

=====

2. UTP Segment Validation

IEEE 802.3 Ethernet copper channel/segment length is commonly limited to **100 m** for the standard Ethernet structured-cabling channel. This function validates the supplied length.

2_validate_segment.py

```
def validate_segment(length_m):
```

```
    """
```

```
    Validate a UTP cable segment.
```

```
    Returns:
```

```
    True -> segment is within 100 m
```

```
    False -> segment exceeds 100 m
```

```
    """
```

```
    if not isinstance(length_m, (int, float)):
```

```
        print("Error: Cable length must be a number.")
```

```
        return False
```

```
    if length_m < 0:
```

```
        print("Error: Cable length cannot be negative.")
```

```
        return False
```

```
    if length_m <= 100:
```

```
        print(f"Valid: {length_m} m is within the 100 m limit.")
```

```
        return True
```

```
    else:
```

```
        print(f"Error: {length_m} m exceeds the 100 m UTP limit.")
```

```
        return False
```

At least 3 test cases

```
print("TEST CASE 1")  
validate_segment(50)
```

```
print("\nTEST CASE 2")  
validate_segment(100)
```

```
print("\nTEST CASE 3")  
validate_segment(120)
```

```
print("\nTEST CASE 4")  
validate_segment(-10)
```

Expected Output

TEST CASE 1

Valid: 50 m is within the 100 m limit.

TEST CASE 2

Valid: 100 m is within the 100 m limit.

TEST CASE 3

Error: 120 m exceeds the 100 m UTP limit.

TEST CASE 4

Error: Cable length cannot be negative.

3. CSMA/CD Simulation with Collision and Back-off Logging

This program simulates 4 hosts attempting to transmit. When multiple hosts transmit simultaneously, a collision is detected and a random binary exponential back-off interval is selected.

The events are stored in:

csma_log.txt

3_csma_cd.py

import random

import time

LOG_FILE = "csma_log.txt"

def log_event(message):

"""Write an event to the log file and display it."""

print(message)

with open(LOG_FILE, "a") as file:

file.write(message + "\n")

def simulate_csma_cd(hosts, rounds=10):

"""Simulate CSMA/CD for the given hosts."""

Clear previous log

with open(LOG_FILE, "w") as file:

file.write("==== CSMA/CD LOG ====\n")

collision_count = {host: 0 for host in hosts}

```
for round_no in range(1, rounds + 1):
```

```
    log_event(f"\nRound {round_no}")
```

```
    # Randomly decide which hosts want to transmit
```

```
    transmitting_hosts = [
```

```
        host for host in hosts if random.choice([True, False])
```

```
    ]
```

```
    if len(transmitting_hosts) == 0:
```

```
        log_event("No host is transmitting.")
```

```
        continue
```

```
    if len(transmitting_hosts) == 1:
```

```
        host = transmitting_hosts[0]
```

```
        log_event(f'{host}: Transmission successful.')  
    else:
```

```
        log_event(
```

```
            f'Collision detected between: "
```

```
            f'{', '.join(transmitting_hosts)}'"
```

```
        )
```

```
    for host in transmitting_hosts:
```

```
        collision_count[host] += 1
```

```
    # Binary exponential backoff
```

```
    k = min(collision_count[host], 10)
```

```
    backoff_slots = random.randint(0, (2 ** k) - 1)
```

```
    log_event(
```



```
f"{host}: Back-off = "  
f"{backoff_slots} slot(s)"  
)
```

```
log_event("\n===== SIMULATION COMPLETE =====")
```

```
# Four hosts
```

```
hosts = ["HOST1", "HOST2", "HOST3", "HOST4"]
```

```
simulate_csma_cd(hosts, rounds=10)
```

```
print("\nCSMA/CD simulation completed.")
```

```
print("Events saved to csma_log.txt")
```

```
Example csma_log.txt
```

```
===== CSMA/CD LOG =====
```

Round 1

Collision detected between: HOST1, HOST3

HOST1: Back-off = 1 slot(s)

HOST3: Back-off = 0 slot(s)

Round 2

HOST4: Transmission successful.

Round 3

Collision detected between: HOST2, HOST3, HOST4

HOST2: Back-off = 2 slot(s)

HOST3: Back-off = 3 slot(s)

HOST4: Back-off = 1 slot(s)



4. Client-Server Star Switch Using socket

This program models:

HOST1

\

\

SWITCH

/

/

HOST2

The server represents the **central switch**. Two clients represent two hosts.

The switch maintains a simple MAC address table and prints it after each frame transmission.

Server / Switch

4_switch_server.py

```
import socket
```

```
HOST = "127.0.0.1"
```

```
PORT = 5000
```

```
# Simulated MAC address table
```

```
mac_table = {}
```

```
def print_mac_table():
```

```
    print("\n===== MAC ADDRESS TABLE =====")
```

```
    if not mac_table:
```

```
        print("Table is empty.")
```

```
    else:
```

```
        for mac, port in mac_table.items():
```

```
print(f"MAC: {mac} -> Port: {port}")
```

```
print("=====\n")
```

```
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
```

```
server_socket.bind((HOST, PORT))
```

```
server_socket.listen(2)
```

```
print("Star Switch started.")
```

```
print(f"Listening on {HOST}:{PORT}")
```

```
connections = []
```

```
try:
```

```
    while len(connections) < 2:
```

```
        connection, address = server_socket.accept()
```

```
        host_id = connection.recv(1024).decode()
```

```
        mac_address = connection.recv(1024).decode()
```

```
        connections.append((connection, host_id))
```

```
        mac_table[mac_address] = host_id
```

```
        print(f"{host_id} connected from {address}")
```

```
print_mac_table()
```

```
print("Two hosts are connected.")
```

```
print("Waiting for frames...\n")
```

```
while True:
```

```
    for connection, source_host in connections:
```

```
        connection.settimeout(0.2)
```

```
        try:
```

```
            frame = connection.recv(1024)
```

```
            if not frame:
```

```
                continue
```

```
            message = frame.decode()
```

```
            print(
```

```
                f'Frame received from {source_host}: "
```

```
                f'{message}'"
```

```
            )
```

```
            # Forward to all other ports
```

```
            for other_connection, other_host in connections:
```

```
                if other_connection != connection:
```

```
                    other_connection.send(
```

```
                        f'Forwarded by SWITCH: {message}'.encode()
```

```
                    )
```

5. Star Switch Class

This program implements:

- add_port(host_id)
- remove_port(host_id)
- broadcast(frame)

The broadcast operation sends the frame to every port **except the source port** and logs every delivery.

5_star_switch.py

```
class StarSwitch:
```

```
    """Simulates a basic Ethernet star switch."""
```

```
    def __init__(self):
```

```
        self.ports = {}
```

```
    def add_port(self, host_id):
```

```
        """Add a host to the switch."""
```

```
        if host_id in self.ports:
```

```
            print(f"Error: {host_id} already exists.")
```

```
            return
```

```
        self.ports[host_id] = []
```

```
        print(f"Port added for {host_id}.")
```

```
    def remove_port(self, host_id):
```

```
        """Remove a host from the switch."""
```

```
        if host_id not in self.ports:
```

```
            print(f"Error: {host_id} does not exist.")
```

```
            return
```

```
log = (  
    f"Frame delivered from "  
    f"{source} -> {host_id}: {data}"  
)
```

```
print(log)
```

```
# -----  
# Testing  
# -----
```

```
switch = StarSwitch()
```

```
# Add hosts
```

```
switch.add_port("HOST1")
```

```
switch.add_port("HOST2")
```

```
switch.add_port("HOST3")
```

```
switch.add_port("HOST4")
```

```
# Broadcast frame from HOST1
```

```
frame1 = {
```

```
    "source": "HOST1",
```

```
    "data": "Hello from HOST1"
```

```
}
```

```
switch.broadcast(frame1)
```

```
# Broadcast another frame
```

```
frame2 = {
```

```
"source": "HOST3",  
"data": "Network test message"  
}
```

```
switch.broadcast(frame2)
```

```
# Remove a host
```

```
switch.remove_port("HOST4")
```

```
# Try broadcasting again
```

```
frame3 = {  
    "source": "HOST2",  
    "data": "HOST4 has been disconnected"  
}
```

```
switch.broadcast(frame3)
```

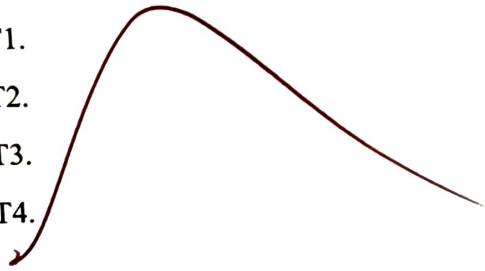
Expected Output

Port added for HOST1.

Port added for HOST2.

Port added for HOST3.

Port added for HOST4.



Frame received from HOST1: Hello from HOST1

Broadcasting frame...

Frame delivered from HOST1 -> HOST2: Hello from HOST1

Frame delivered from HOST1 -> HOST3: Hello from HOST1

Frame delivered from HOST1 -> HOST4: Hello from HOST1

Frame received from HOST3: Network test message

Broadcasting frame...

Frame delivered from HOST3 -> HOST1: Network test message

Frame delivered from HOST3 -> HOST2: Network test message

Frame delivered from HOST3 -> HOST4: Network test message

Port removed for HOST4.

Frame received from HOST2: HOST4 has been disconnected

Broadcasting frame...

Frame delivered from HOST2 -> HOST1: HOST4 has been disconnected

Frame delivered from HOST2 -> HOST3: HOST4 has been disconnected

